

# TRAITEMENT AUTOMATIQUE DU LANGAGE NATUREL

Résumé Complet du Cours – Préparation Examen

Master IA – FSSM – J.ZAHIR – 2026

## Table des matières

|          |                                                      |           |          |                                                                       |           |
|----------|------------------------------------------------------|-----------|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Vectorisation du Texte (Session 1)</b>            | <b>1</b>  |          |                                                                       |           |
| 1.1      | Bag of Words (BoW)                                   | 1         | 4.4      | Techniques de Lissage (Smoothing)                                     | 6         |
| 1.1.1    | Tokenisation                                         | 1         | 4.4.1    | Interpolation et Backoff                                              | 6         |
| 1.1.2    | n-grammes : capture du contexte local                | 1         | 4.4.2    | Good-Turing Smoothing                                                 | 6         |
| 1.1.3    | Limites critiques de BoW                             | 1         | 4.5      | Évaluation des Modèles de Langue                                      | 6         |
| 1.2      | TF-IDF (Term Frequency – Inverse Document Frequency) | 2         | <b>5</b> | <b>Plongements Lexicaux (Word Embeddings)</b>                         | <b>7</b>  |
| 1.3      | Normalisation et Prétraitement du Texte              | 2         | 5.1      | Motivations et Limites du One-Hot                                     | 7         |
| <b>2</b> | <b>Tâches NLP et Paradigmes d'Apprentissage</b>      | <b>2</b>  | 5.2      | Word Embeddings – Avantages                                           | 7         |
| 2.1      | Taxonomie des Tâches NLP                             | 2         | 5.3      | Word2Vec (Mikolov et al., Google 2013)                                | 7         |
| 2.1.1    | Par niveau linguistique                              | 2         | 5.3.1    | Étapes Skip-gram                                                      | 7         |
| 2.1.2    | Par architecture (mécanisme de sortie)               | 2         | 5.4      | GloVe (Pennington et al., 2014)                                       | 7         |
| 2.1.3    | Sequence Labeling – Many-to-Many                     | 2         | 5.5      | FastText (Joulin et al., Facebook 2016)                               | 8         |
| 2.1.4    | Schémas d'étiquetage BIO et BIOES                    | 3         | 5.5.1    | Synthèse des représentations                                          | 8         |
| 2.2      | Paradigmes d'Apprentissage                           | 3         | <b>6</b> | <b>Modèles de Langues Neuronales – De la Récurrence à l'Attention</b> | <b>8</b>  |
| 2.2.1    | SSL – Tâches Prétexte                                | 3         | 6.1      | RNN – Réseaux de Neurones Récurrents                                  | 8         |
| 2.2.2    | Contrastive Learning                                 | 3         | 6.2      | LSTM – Long Short-Term Memory                                         | 8         |
| 2.2.3    | RLHF – Processus en 3 étapes                         | 3         | 6.3      | La Révolution du Transformer (Vaswani et al., 2017)                   | 8         |
| <b>3</b> | <b>Analyse de Sentiment</b>                          | <b>4</b>  | 6.3.1    | Encodeur vs Décodeur                                                  | 8         |
| 3.1      | Fondamentaux                                         | 4         | 6.3.2    | Input Embedding + Positional Encoding                                 | 9         |
| 3.2      | Approche Lexicon-based                               | 4         | 6.3.3    | Self-Attention (Auto-Attention)                                       | 9         |
| 3.2.1    | Approche Dictionnaire (propagation sur WordNet)      | 4         | 6.3.4    | Multi-Head Attention (MHA)                                            | 9         |
| 3.2.2    | Algorithme de Turney (approche corpus)               | 4         | 6.4      | RAG – Retrieval-Augmented Generation                                  | 9         |
| 3.2.3    | SentiWordNet                                         | 4         | 6.5      | Fine-tuning des Modèles                                               | 10        |
| 3.3      | Deep Learning pour le Sentiment                      | 4         | <b>7</b> | <b>Métriques d'Évaluation en TALN</b>                                 | <b>10</b> |
| 3.3.1    | RNN/LSTM                                             | 4         | 7.1      | Classification et Labellisation                                       | 10        |
| 3.3.2    | Transformers (BERT)                                  | 4         | 7.2      | NER : Biais d'évaluation                                              | 10        |
| 3.4      | ABSA – Aspect-Based Sentiment Analysis               | 5         | 7.2.1    | Standard CoNLL (Strict)                                               | 10        |
| 3.5      | XAI – Explainability pour l'Analyse de Sentiment     | 5         | 7.2.2    | Standard MUC (Flexible)                                               | 10        |
| <b>4</b> | <b>Modèles de Langage Probabilistes (N-grammes)</b>  | <b>5</b>  | 7.3      | Métriques Lexicales (Traduction/Résumé)                               | 11        |
| 4.1      | Langue comme ensemble de règles (avant 1980)         | 5         | 7.3.1    | BLEU (Bilingual Evaluation Understudy)                                | 11        |
| 4.2      | Langue comme ensemble de probabilités                | 5         | 7.3.2    | ROUGE (Recall-Oriented Understudy for Gisting Evaluation)             | 11        |
| 4.3      | N-grammes et Modèles de Langue                       | 5         | 7.4      | Métriques Sémantiques (Embeddings)                                    | 11        |
| 4.3.1    | Problème des N-grammes à probabilité zéro            | 6         | 7.4.1    | BERTScore                                                             | 11        |
|          |                                                      |           | 7.4.2    | COMET et BLEURT                                                       | 11        |
|          |                                                      |           | 7.5      | Évaluation des LLMs                                                   | 12        |
|          |                                                      |           | 7.5.1    | Benchmarks standardisés                                               | 12        |
|          |                                                      |           | 7.5.2    | LLM-as-a-Judge                                                        | 12        |
|          |                                                      |           | 7.5.3    | Sécurité, Véridicité et NLI                                           | 12        |
| <b>8</b> | <b>Récapitulatif Synthétique – Fiches Mémo</b>       | <b>12</b> |          |                                                                       |           |

# 1 Vectorisation du Texte (Session 1)

## 1.1 Bag of Words (BoW)

### Définition – Bag of Words

Représentation qui transforme des documents en **vecteurs de longueur fixe**. Elle extrait des features numériques à partir de texte non structuré pour les modèles ML.

- **Ignore** l'ordre des mots et la grammaire (le contexte est perdu)
- Se concentre sur la **fréquence** d'occurrence
- Résultat : une matrice Document-Terme de haute dimension (sparse)

**Processus** : Définir un Vocabulaire  $V \rightarrow$  Compter les occurrences de chaque  $v \in V$ .

### 1.1.1 Tokenisation

- **Token** : unité de base d'analyse (en NLP, les «mots» sont des tokens)
- **Tokenisation** : découpage du flux de caractères en unités significatives
- **Normalisation** : lowercasing, stemming/lemmatisation
- **Stop words** : mots très fréquents (le, un, est...) souvent supprimés

Niveaux de granularité :

| Niveau    | Exemple               | Avantage/Inconvénient                   |
|-----------|-----------------------|-----------------------------------------|
| Mot       | ["NLP", "est", "fun"] | Sens préservé / grand vocab + OOV       |
| Sous-mot  | ["Token", "ization"]  | Efficace, devine les mots nouveaux      |
| Caractère | ["T", "o", "k"...]    | Zéro OOV / perd sens, séquences longues |

### 1.1.2 n-grammes : capture du contexte local

#### n-grammes

Séquence contiguë de  $n$  items (mots ou caractères). La fenêtre glissante de taille  $n$  se déplace sur la phrase.

- $n = 1$  : Unigramme (= BoW standard)
- $n = 2$  : Bigramme     $n = 3$  : Trigramme
- **Avantage** : préserve l'ordre local, capture les expressions multi-mots («pas content»)
- **Inconvénient** : explosion combinatoire du vocabulaire pour  $n$  grand

### 1.1.3 Limites critiques de BoW

#### Limitations

1. **Perte de contexte & d'ordre** : «Le chien mord l'homme» = «L'homme mord le chien»
2. **Sparse** : vecteurs remplis de zéros
3. **Malédiction de la dimensionnalité** : vocab 50k mots  $\Rightarrow$  espace 50k dimensions
4. **Pas de similarité sémantique** : «moto» et «vélo» vus comme totalement différents
5. **Polysémie** : «avocat» (légume vs juriste) indistinguable

## 1.2 TF-IDF (Term Frequency – Inverse Document Frequency)

### TF-IDF – Intuition

Évalue l'importance d'un mot pour un document dans une collection.

- **TF élevé + IDF élevé** : mot fréquent dans CE document, rare ailleurs  $\Rightarrow$  mot clé
- **TF élevé + IDF faible** : mot fréquent partout («le», «est»)  $\Rightarrow$  peu informatif

## Formules TF-IDF

$$\text{TF}(w, d) = \frac{\text{occurrences de } w \text{ dans } d}{\text{total des termes dans } d}$$

$$\text{IDF}(w, D) = \log \frac{|D|}{|\{d_j : w \in d_j\}|}$$

$$\text{TF-IDF}(w, d, D) = \text{TF}(w, d) \times \text{IDF}(w, D)$$

Schémas de pondération : Booléen (0/1) → Compteur (fréquence) → TF-IDF (pondéré)

### 1.3 Normalisation et Prétraitement du Texte

#### Stemming vs Lemmatisation

- **Stemming** : suppression de suffixes par heuristiques pour obtenir la «racine» (stem). Ex : wolves → wolv, cats → cat. **Porter's Stemmer** = stemmer anglais le plus commun.
- **Lemmatisation** : retourne la forme canonique du dictionnaire (lemme). Utilise un vocabulaire et une analyse morphologique. Ex : am/are/is → be, wolves → wolf
- **Différence clé** : le stemming est plus rapide mais moins précis ; la lemmatisation est linguistiquement correcte

## 2 Tâches NLP et Paradigmes d'Apprentissage

### 2.1 Taxonomie des Tâches NLP

#### 2.1.1 Par niveau linguistique

| Niveau                   | Tâches                                        |
|--------------------------|-----------------------------------------------|
| Phonétique/Morphologique | Stemming, Lemmatisation, Transcription parole |
| Syntaxique               | POS Tagging, Chunking, Parsing                |
| Sémantique               | NER, WSD (désambiguïsation)                   |
| Pragmatique/Discours     | Analyse de sentiment, Résolution coréférences |

#### 2.1.2 Par architecture (mécanisme de sortie)

| Architecture    | Sortie                          | Exemple              |
|-----------------|---------------------------------|----------------------|
| Many-to-One     | Un label pour toute la séquence | Sentiment, Intent    |
| Many-to-Many    | Un label par token (synchrone)  | NER, POS Tagging     |
| Span Extraction | Deux indices (début, fin)       | QA extractif (SQuAD) |
| Seq2Seq         | Nouvelle séquence (asynchrone)  | Traduction, Résumé   |

#### Labeling vs Span Extraction

**Labeling** : N décisions (une par mot). **Span Extraction** : 2 pointeurs dans le texte source (« pointage »).

#### 2.1.3 Sequence Labeling – Many-to-Many

$|X| = |Y|$  (sortie de même longueur que l'entrée)

- **POS Tagging** : étiquetage morphosyntaxique (Nom, Verbe, Adj...)
- **NER** : identification d'entités (Personnes, Lieux, Organisations)
- **Chunking** : segmentation en groupes syntaxiques (NP, VP)
- **SRL** : rôles sémantiques (Qui fait quoi ?)

### 2.1.4 Schémas d'étiquetage BIO et BIOES

| Token        | BIO   | BIOES |
|--------------|-------|-------|
| République   | B-LOC | B-LOC |
| Démocratique | I-LOC | I-LOC |
| du           | I-LOC | I-LOC |
| Congo        | I-LOC | E-LOC |
| (Paris seul) | B-LOC | S-LOC |

**BIOES** réduit les erreurs de segmentation en forçant le modèle à prédire explicitement la fin d'une entité.

## 2.2 Paradigmes d'Apprentissage

### Les 4 Paradigmes

1. **Supervisé** : couples  $(X, Y)$  annotés par l'humain
2. **Non-supervisé** : clustering, topic detection (sans labels)
3. **Auto-supervisé (Self-Supervised)** : labels extraits des données elles-mêmes
4. **RLHF** : alignement sur les préférences humaines (utilité, sécurité)

### 2.2.1 SSL – Tâches Prétexste

- **MLM – Masked Language Modeling (BERT)** : masque 15% des mots et les prédit  $\Rightarrow$  sémantique bidirectionnelle
- **CLM – Causal Language Modeling (GPT)** : prédit le mot suivant  $\Rightarrow$  génération de texte
- **NSP – Next Sentence Prediction (BERT)** : classification binaire, la phrase B suit-elle A ?

| Pré-entraînement       | Tâche en aval (Downstream)     |
|------------------------|--------------------------------|
| Auto-supervision (MLM) | Classification / NER           |
| Auto-supervision (CLM) | Chatbots / Résumé / Traduction |

### 2.2.2 Contrastive Learning

#### Principe

Rapprocher les exemples similaires (positifs) et éloigner les exemples différents (négatifs). Application phare : **Word2Vec (Skip-gram + Negative Sampling)**.

#### Fonction de perte (simplifiée)

$$\mathcal{L} = -\log(\sigma(v_{\text{pos}} \cdot v_{\text{target}})) - \sum \log(\sigma(-v_{\text{neg}} \cdot v_{\text{target}}))$$

### 2.2.3 RLHF – Processus en 3 étapes

1. **SFT (Supervised Fine-Tuning)** : entraînement supervisé sur données haute qualité
2. **Reward Modeling (RM)** : modèle de score qui apprend à classer deux réponses ( $y_w$  gagnante vs  $y_l$  perdante).  
Perte :  $\mathcal{L} = -\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))$
3. **Optimisation (PPO/DPO)** : ajuster le modèle pour maximiser la récompense tout en restant proche du modèle initial

#### Objectifs HHH

**Helpfulness** (Utilité) · **Honesty** (Honnêteté) · **Harmlessness** (Innocuité)

### 3 Analyse de Sentiment

#### 3.1 Fondamentaux

##### Niveaux d'analyse

- **Document, Phrase, Aspect (ABSA)**
- **Polarité** : positif, négatif, neutre
- **Émotions (Ekman)** : Joie, Colère, Peur, Dégoût, Tristesse, Surprise
- **Subjectivité vs Objectivité** : opinion («écran trop petit») vs fait («écran 6 pouces»)

##### Défis sémantiques

- **Négation** : «Le film n'est pas bon mais l'acteur est super»
- **Litote** : «Le film n'était pas mauvais»  $\neq$  «Le film était bon» (atténuation)
- **Ironie/Sarcasme** : difficile à détecter pour les modèles classiques

#### 3.2 Approche Lexicon-based

##### Hypothèse

La polarité globale est une fonction des polarités individuelles des mots.

##### 3.2.1 Approche Dictionnaire (propagation sur WordNet)

1. **Seed words** : excellent(+), horrible(-) – polarité évidente
2. **Propagation** : synonyme  $\Rightarrow$  même polarité, antonyme  $\Rightarrow$  polarité inverse
3. **Itération** jusqu'à 0 nouveaux mots ajoutés

##### 3.2.2 Algorithme de Turney (approche corpus)

1. Extraction de locutions (Adjectif + Nom) via POS-tagging
2. Calcul de l'Orientation Sémantique (SO) via la mesure **PMI**
3. Classification du document par la moyenne des scores SO

##### PMI – Pointwise Mutual Information

$$\text{PMI}(w_1, w_2) = \log_2 \frac{P(w_1, w_2)}{P(w_1)P(w_2)}$$

PMI  $\approx 0$  : indépendants · PMI  $> 0$  : corrélés · PMI  $< 0$  : exclusion

Si  $\text{PMI}(p, \text{"excellent"}) > \text{PMI}(p, \text{"poor"})$  : la locution  $p$  est probablement positive.

##### 3.2.3 SentiWordNet

Chaque entrée WordNet  $\Rightarrow$  trois scores : Positivité, Négativité, Objectivité. Nécessite la WSD (désambiguïsation) avant application des scores.

### 3.3 Deep Learning pour le Sentiment

#### 3.3.1 RNN/LSTM

- Utilisent des embeddings denses (Word2Vec/GloVe) capturant la sémantique
- Maintiennent un «état mémoire» du début à la fin de la phrase
- Gèrent les dépendances à long terme
- **Limites** : gradient vanishing, séquentialité (impossible de paralléliser)

#### 3.3.2 Transformers (BERT)

- **Mécanisme d'Attention** : focus sélectif sur les mots pertinents pour la polarité
- **Contextualisation** : «terrible» = positif dans «talent terrible», négatif dans «service terrible»
- **Fine-tuning** : modèle pré-entraîné + couche softmax pour la classification de sentiment

| Critère             | Lexique/ML classique | BERT                |
|---------------------|----------------------|---------------------|
| Négation complexe   | Faible               | Excellente          |
| Sarcasme/Ironie     | Quasi-nulle          | Modérée             |
| Besoin de données   | Faible               | Élevé (Fine-tuning) |
| Vitesse d'inférence | Très rapide          | Lente (GPU)         |

### 3.4 ABSA – Aspect-Based Sentiment Analysis

#### ABSA – Analyse Granulaire

Extraire le quintuple (**entité, aspect, sentiment, opinion holder, temps**).

Exemple : «Le son est cristallin (+), mais le design est daté (-).»

→ L'analyse globale donnerait «Neutre», l'ABSA donne deux informations riches.

— **ATE (Aspect Term Extraction)** : identifier «Batterie», «Prix», «Écran» ⇒ BIO Tagging

— **ASC (Aspect Sentiment Classification)** : polarité spécifique à chaque aspect

#### Target-Dependent Attention

$$\text{Attention}(Q_{\text{aspect}}, K_{\text{sentence}}, V_{\text{sentence}})$$

L'aspect sert de **Query** : le modèle ignore les adjectifs ne concernant pas l'aspect cible.

**Joint Learning vs Pipeline** : entraîner un modèle unique pour ATE + ASC simultanément, évitant la propagation d'erreurs.

### 3.5 XAI – Explainability pour l'Analyse de Sentiment

— **SHAP** : attribution de valeur par mot

— **LIT (Learning Interpretability Tool)** : visualisation des têtes d'attention, analyse «What-if»

— **Saliency Maps** : quels mots ont le plus influencé la sortie ?

## 4 Modèles de Langage Probabilistes (N-grammes)

### 4.1 Langage comme ensemble de règles (avant 1980)

— **Paradigme symbolique** : règles formelles, grammaires hors-contexte (CFG)

— CFG : tuple  $\langle N, \Sigma, S, R \rangle$  (non-terminaux, terminaux, symbole de départ, règles)

— **Limite** : trop rigide, ne gère pas le bruit, la polysémie, les erreurs

### 4.2 Langage comme ensemble de probabilités

— **Al-Kindi (IXe s.)** : analyse de fréquence des lettres pour la cryptanalyse

— **Markov (1913)** : probabilité d'un symbole dépend du précédent

— **Shannon (1948)** : entropie, langage comme source stochastique, redondance ~50-75%

— **IBM (années 80)** : HMM pour la reconnaissance vocale, modèle du canal bruité

#### Entropie de Shannon

$$H = - \sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

Entropie élevée = bruit aléatoire · Entropie faible = langage prévisible

### 4.3 N-grammes et Modèles de Langue

#### Modèle de Langue (LM)

Assigne des probabilités à des séquences de mots. Calcule  $P(w \mid h)$  ou  $P(w_1, \dots, w_k)$ .

**Règle de la chaîne + Hypothèse de Markov**

$$P(w_1, \dots, w_n) = \prod_{k=1}^n P(w_k | w_1, \dots, w_{k-1}) \approx \prod_{k=1}^n P(w_k | w_{k-1})$$

**MLE (Maximum Likelihood Estimation) pour les bigrammes :**

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

**En pratique :** les trigrammes offrent le meilleur équilibre approximation/complexité.

**4.3.1 Problème des N-grammes à probabilité zéro**

Tout  $n$ -gramme absent du corpus reçoit  $P = 0 \Rightarrow$  la probabilité de toute phrase le contenant = 0.

**Vocabulaire ouvert :** modéliser les mots inconnus avec <UNK>.

**4.4 Techniques de Lissage (Smoothing)****Pourquoi le lissage ?**

Ce qu'on n'a pas vu à l'entraînement n'est pas impossible ! Le lissage alloue de la masse de probabilité aux événements rares/non vus.

**Laplace (Add-1)**

$$P_{\text{Lap}}(w_i | w_{i-1}) = \frac{1 + C(w_{i-1}, w_i)}{V + C(w_{i-1})}$$

**Limite :** donne trop de masse aux événements non vus.

**Lidstone (Add- $\lambda$ )**

$$P_{\text{Lid}}(w_n | w_1, \dots, w_{n-1}) = \frac{\lambda + C(w_1, \dots, w_n)}{\lambda V + C(w_1, \dots, w_{n-1})}, \quad 0 < \lambda \leq 1$$

**4.4.1 Interpolation et Backoff****Interpolation linéaire (Jelinek-Mercer)**

Combiner trigramme, bigramme et unigramme pondérés par des  $\lambda_i$  :

$$\hat{P}(w_n | w_{n-2}, w_{n-1}) = \lambda_1 P(w_n | w_{n-2}, w_{n-1}) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n)$$

avec  $\sum_i \lambda_i = 1$ .

**Backoff (Katz)**

Si le  $n$ -gramme a un compte nul, on «recule» vers un modèle d'ordre inférieur (en escomptant les probabilités pour maintenir une distribution valide).

**4.4.2 Good-Turing Smoothing**

**Idée :** utiliser le compte des événements vus une fois pour estimer le compte des événements jamais vus.  $c^* \approx (c - 0.75)$  (Absolute Discounting).

**4.5 Évaluation des Modèles de Langue**

- **Évaluation extrinsèque :** intégrer le LM dans une application et mesurer l'amélioration (coûteux)
- **Évaluation intrinsèque :** mesurer la qualité du LM de façon indépendante

**Perplexité**

Mesure à quel point un LM prédit correctement un échantillon.

**Perplexité faible = meilleur modèle.**

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-1/N}$$

**Découpage standard** : 80% entraînement, 10% développement (devset), 10% test.

## 5 Plongements Lexicaux (Word Embeddings)

### 5.1 Motivations et Limites du One-Hot

**Limites du One-Hot Encoding**

- **Vecteurs orthogonaux** : cosinus = 0 entre tous les mots  $\Rightarrow$  pas de similarité sémantique
- **Très haute dimension**  $\mathbb{R}^V$  (millions de dimensions)  $\Rightarrow$  malédiction de la dimension
- **Représentation discrète** : pas de notion de proximité

**Hypothèse Distributionnelle**

«*You shall know a word by the company it keeps*» – J.R. Firth (1957)

Le sens d'un mot est défini par les contextes (voisins) dans lesquels il apparaît.

Le **contexte** = mots dans un rayon  $m$  autour du mot central.

### 5.2 Word Embeddings – Avantages

- Représentations **denses** de dimension  $D$  (100-300) avec  $D \ll |V|$
- Capture la **sémantique** : roi + femme – homme  $\approx$  reine
- Projection TensorFlow : <https://projector.tensorflow.org/>

### 5.3 Word2Vec (Mikolov et al., Google 2013)

**Deux architectures**

- **Skip-gram** : prédit le contexte à partir d'un mot central  $P(C | W)$
- **CBOW** : prédit le mot cible à partir du contexte  $P(W | C)$

#### 5.3.1 Étapes Skip-gram

1. **Vocabulaire** : créer le vocabulaire, attribuer des indices
2. **Fenêtre glissante** : générer des paires (mot cible, mot contexte) pour chaque position
3. **Réseau de neurones** : couche entrée  $\rightarrow$  couche cachée (embeddings, **linéaire**)  $\rightarrow$  softmax
4. **Rétropropagation** : ajuster les vecteurs via descente de gradient
5. **Extraction** : poids de la couche cachée = embeddings finaux

**Probabilité Skip-gram (Softmax)**

$$P(w_{\text{contexte}} | w_{\text{cible}}) = \frac{\exp(v_{\text{contexte}} \cdot v_{\text{cible}})}{\sum_{k=1}^N \exp(v_k \cdot v_{\text{cible}})}$$

**En pratique** : Negative Sampling ou Hierarchical Softmax (trop coûteux pour vocab  $\sim$  1M)

**Negative Sampling**

Apprentissage contrastif : maximiser la probabilité de la paire réelle vs mots aléatoires (bruit). Inspiré de CLIP (OpenAI). Transforme la classification sur  $V$  classes en classification binaire ultra-rapide.

### 5.4 GloVe (Pennington et al., 2014)

Vecteurs globaux basés sur les statistiques de co-occurrence dans tout le corpus.



## 5.5 FastText (Joulin et al., Facebook 2016)

### Innovation : représentation par sous-mots

Décompose chaque mot en  $n$ -grams de caractères ( $n \in \{3, 4, 5, 6\}$ ).

**Exemple** : apple  $\rightarrow$  <ap, app, ppl, ple, le> + <apple>

$$v(\text{apple}) = v(\text{<ap>}) + v(\text{<app>}) + v(\text{<ppl>}) + v(\text{<ple>}) + v(\text{<le>}) + v(\text{<apple>})$$

| Critère                | Word2Vec              | FastText                  |
|------------------------|-----------------------|---------------------------|
| Représentation         | Mot entier (one-hot)  | Somme des $n$ -grams      |
| Mots OOV               | Impossible (KeyError) | Possible (via $n$ -grams) |
| Langues morphologiques | Inefficace            | Efficace                  |

### 5.5.1 Synthèse des représentations

- **One-Hot** : granularité = mot atomique (aucune relation)
- **Word2Vec** : granularité = mot sémantique (voisinage)
- **FastText** : granularité = sous-mots/caractères (morphologie)

**ELMo (2018)** : premiers embeddings **contextualisés** (Bi-LSTM). Actuellement, les embeddings reposent sur les Transformers (BERT, GPT...).

## 6 Modèles de Langues Neuronaux – De la Récurrence à l'Attention

### 6.1 RNN – Réseaux de Neurones Récurents

#### Équations RNN

$$h_t = \phi(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

$h_t$  = état caché (mémoire à court terme),  $\phi = \tanh$

#### Limites des RNN

- **Gradient vanishing** : difficulté à retenir l'information sur de longs textes
- **Séquentialité** : impossible de paralléliser  $\Rightarrow$  lent pour le Big Data
- BPTT (Backpropagation Through Time) requis pour l'entraînement

### 6.2 LSTM – Long Short-Term Memory

#### Les 3 portes + cellule d'état

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Oubli})$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Entrée})$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Sortie})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Mise à jour cellule})$$

L'information peut circuler sur de longues distances sans altération majeure.

**Seq2Seq (2014)** : architecture encodeur-décodeur (LSTM), état de l'art pour traduction.

### 6.3 La Révolution du Transformer (Vaswani et al., 2017)

#### 6.3.1 Encodeur vs Décodeur

| Composant | Encodeur                     | Décodeur                |
|-----------|------------------------------|-------------------------|
| Rôle      | Compréhension (Source)       | Génération (Cible)      |
| Attention | Bidirectionnelle (voit tout) | Masquée (voit le passé) |
| Modèles   | BERT, RoBERTa                | GPT-1/2/3/4             |

### 6.3.2 Input Embedding + Positional Encoding

$$X_{\text{input}} = \text{Embedding} + \text{PositionalEncoding}$$

Le Transformer traite tous les mots **en parallèle**  $\Rightarrow$  nécessite d'encoder la position.

#### Encodage Positionnel Sinusoïdal (Vaswani)

$$PE_{(\text{pos}, 2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(\text{pos}, 2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

Propriété :  $PE_{\text{pos}+k}$  est une fonction linéaire de  $PE_{\text{pos}}$   $\Rightarrow$  le modèle apprend les relations relatives.

### 6.3.3 Self-Attention (Auto-Attention)

#### Q

- **Query (Q)** : «Ce que je cherche» – le mot pose une question
- **Key (K)** : «Ce que je propose» – le mot indique son contenu
- **Value (V)** : «Ce que je transmet» – l'info réelle à propager

#### Calcul de l'Attention (Scaled Dot-Product)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

1. Score :  $Q \cdot K^T$
2. Normalisation :  $/\sqrt{d_k} + \text{softmax}$
3. Agrégation :  $\times V$

### 6.3.4 Multi-Head Attention (MHA)

#### MHA

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Chaque tête capture une relation différente (syntaxe, sémantique...). Parallélisation totale sur GPU.

## 6.4 RAG – Retrieval-Augmented Generation

#### Définition RAG

Architecture étendant les capacités d'un LLM en lui donnant accès à des **données externes**. Au lieu de se fier à sa «mémoire paramétrique», le système interroge une «mémoire externe».

- Réduction des hallucinations
- Mise à jour des connaissances sans ré-entraînement
- Citations des sources

#### Similarité Cosinus

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

#### Étape 1 – Indexation :

1. **Chunking** : découpage des documents (ex : 500 tokens)
2. **Embedding** : conversion en vecteur via Bi-Encoder (BERT, Ada-002)
3. **Stockage** : dans un Vector Store (Pinecone, Milvus, Weaviate)

#### Étape 2 – Récupération & Génération :

1. Vectorisation de la question :  $v_q$
2. Extraction des  $k$  segments les plus proches
3. Augmentation du prompt + Génération par le LLM

## 6.5 Fine-tuning des Modèles

| Méthode             | Description                    | Usage                        |
|---------------------|--------------------------------|------------------------------|
| Fine-tuning complet | Tous les paramètres mis à jour | Performance max / coût élevé |
| PEFT (LoRA, etc.)   | < 1% des paramètres entraînés  | Efficace en mémoire          |
| Few-shot prompting  | Pas de mise à jour des poids   | Prototypage rapide           |
| Chain-of-Thought    | Raisonnement étape par étape   | Arithmétique, logique        |

### LoRA (Low-Rank Adaptation)

Hypothèse : la mise à jour des poids  $\Delta W$  a un «rang intrinsèque» faible.

Au lieu de modifier  $W \in \mathbb{R}^{d \times k}$ , on injecte deux matrices bas rang :  $h = W_0 x + B A x$ .

Seules  $A$  et  $B$  sont entraînées ;  $W_0$  est gelé  $\Rightarrow$  gain mémoire massif.

### Chain of Thought (CoT)

Standard : Entrée  $\rightarrow$  Réponse

CoT : Entrée  $\rightarrow$  Raisonnement (rationales)  $\rightarrow$  Réponse

**STaR** : le modèle génère des CoT, filtre les corrects, se fine-tune dessus (itératif).

**Propriété émergente** : le CoT n'apparaît que sur des modèles > 7B paramètres.

## 7 Métriques d'Évaluation en TALN

### 7.1 Classification et Labellisation

#### Précision

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{(\beta^2 \cdot P) + R}$$

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} = \frac{2TP}{2TP + FP + FN}$$

### 7.2 NER : Biais d'évaluation

#### Évaluation au niveau Token – Biais

- **Sur-évaluation** : une entité tronquée («CHU» au lieu de «CHU Mohammed VI») donne un VP au premier token, gonflant artificiellement les métriques
- **Sévérité excessive** (exact-match binaire) : pénalise une extraction partielle aussi lourdement qu'une absence totale

#### 7.2.1 Standard CoNLL (Strict)

Pour qu'une entité soit TP : frontières exactes ET type exact.

$$\begin{cases} \text{Token}_{\text{début}, \text{pred}} = \text{Token}_{\text{début}, \text{ref}} \text{ et } \text{Token}_{\text{fin}, \text{pred}} = \text{Token}_{\text{fin}, \text{ref}} \\ \text{Classe}_{\text{pred}} = \text{Classe}_{\text{ref}} \end{cases}$$

#### 7.2.2 Standard MUC (Flexible)

Décompose en : **COR** (correct) + **PAR** (partiel) + **INC** (incorrect)

$$P_{\text{MUC}} = \frac{\text{COR} + 0.5 \times \text{PAR}}{\text{Total prédictions}}, \quad R_{\text{MUC}} = \frac{\text{COR} + 0.5 \times \text{PAR}}{\text{Total références}}$$

| Prédiction                          | CoNLL | MUC                                |
|-------------------------------------|-------|------------------------------------|
| [CHU Mohammed VI] LOC correct       | TP    | Score max                          |
| [CHU Mohammed VI] PER (type erroné) | FP+FN | Partiel (frontière OK, type inc.)  |
| [CHU] LOC (tronqué)                 | FP+FN | Partiel (type OK, frontière part.) |
| [Université de Marrakech] LOC       | FP+FN | INC                                |

## 7.3 Métriques Lexicales (Traduction/Résumé)

### 7.3.1 BLEU (Bilingual Evaluation Understudy)

#### BLEU – Orienté Précision

Précision des  $n$ -grammes, corrigée pour éviter le sur-comptage.

#### BLEU – Formule

$$p_n = \frac{\sum_C \sum_{n\text{-gram} \in C} \text{Count}_{\text{clip}}(n\text{-gram})}{\sum_C \sum_{n\text{-gram} \in C} \text{Count}(n\text{-gram})}$$

$$BP = \begin{cases} 1 & \text{si } c > r \\ e^{(1-r/c)} & \text{si } c \leq r \end{cases} \quad \text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

$N = 4$ ,  $w_n = 1/N$  généralement. La BP pénalise les traductions trop courtes.

#### Exemple BLEU-2 :

- Référence : «le chat est sur le tapis» ( $r = 6$ )
- Hypothèse : «le chat le chat est sur tapis» ( $c = 7$ )
- $p_1 = 6/7 \approx 0.857$ ,  $p_2 = 3/6 = 0.5$ ,  $BP = 1$
- $\text{BLEU}_2 = \sqrt{0.857 \times 0.5} \approx 0.655$  (65.5%)

### 7.3.2 ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

#### ROUGE – Orienté Rappel (Résumé)

- **ROUGE-N** : rappel des  $n$ -grammes

$$\text{ROUGE-N} = \frac{\sum_R \sum_{n\text{-gram} \in R} \text{Count}_{\text{match}}(n\text{-gram})}{\sum_R \sum_{n\text{-gram} \in R} \text{Count}(n\text{-gram})}$$

- **ROUGE-L** : Plus Longue Sous-Séquence Commune (LCS) – similarité de structure sans exiger la contiguïté

**Exemple :** Référence «le chat est sur le tapis» ( $|R| = 6$ ), Hypothèse «le chat est assis» ( $|H| = 4$ ) :

| Métrique        | Rappel       | Précision    | F1   |
|-----------------|--------------|--------------|------|
| ROUGE-1         | $3/6 = 0.50$ | $3/4 = 0.75$ | 0.60 |
| ROUGE-2         | $2/5 = 0.40$ | $2/3 = 0.67$ | 0.50 |
| ROUGE-L (LCS=3) | $3/6 = 0.50$ | $3/4 = 0.75$ | 0.60 |

#### Limites des métriques lexicales

Un synonyme parfait non présent dans la référence sera lourdement pénalisé. Absence totale de prise en compte de la sémantique.

## 7.4 Métriques Sémantiques (Embeddings)

### 7.4.1 BERTScore

Utilise les représentations contextualisées de BERT pour calculer la **similarité cosinus** entre chaque token généré et chaque token de référence.

$$R_{\text{BERT}} = \frac{1}{|y|} \sum_{y_j \in y} \max_{x_i \in x} x_i^\top y_j, \quad P_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{y_j \in y} x_i^\top y_j$$

### 7.4.2 COMET et BLEURT

Métriques de régression apprises (Transformers fine-tunés sur des évaluations humaines WMT). Prédissent directement une note de qualité humaine.

## 7.5 Évaluation des LLMs

### 7.5.1 Benchmarks standardisés

| Benchmark          | Ce qu'il évalue                                      |
|--------------------|------------------------------------------------------|
| MMLU               | Connaissances multidisciplinaires (57 domaines), QCM |
| GSM8K / MATH       | Raisonnement mathématique multi-étapes               |
| HumanEval (pass@k) | Complétion de code Python (exécution fonctionnelle)  |

### 7.5.2 LLM-as-a-Judge

Utilisation d'un LLM frontière (GPT-4) comme juge. Deux modes :

- **Pairwise Comparison** : juge deux réponses et déclare un vainqueur
- **Single Answer Grading** : note une réponse absolument

#### Biais du Juge Artificiel

- **Position Bias** : favorise la réponse présentée en premier (parade : inverser l'ordre)
- **Verbosity Bias** : associe prolixité à qualité (réponse longue > réponse concise)
- **Self-Enhancement Bias** : note élevée pour les réponses de la même lignée de modèles

### 7.5.3 Sécurité, Véridicité et NLI

#### NLI pour détecter les Hallucinations

Vérification factuelle formalisée comme un problème de logique linguistique :

- **Prémisse (P)** : source de vérité (dossier médical, texte de loi)
- **Hypothèse (H)** : réponse générée par le modèle évalué

Classification par un modèle NLI :

1. **Entailment** : H est logiquement garantie par P  $\Rightarrow$  pas d'hallucination
2. **Contradiction** : H contredit P  $\Rightarrow$  hallucination stricte
3. **Neutral** : H ni confirmée ni infirmée  $\Rightarrow$  hallucination par extrapolation

Taux d'hallucination = proportion de (Contradiction + Neutral).

## 8 Récapitulatif Synthétique – Fiches Mémo

#### Formules Clés à Retenir

**TF-IDF** :  $TF \times \log(|D|/df)$   
**Attention** :  $\text{softmax}(QK^T/\sqrt{d_k})V$   
**LSTM Oubli** :  $f_t = \sigma(W_f[h_{t-1}, x_t])$   
**BLEU** :  $BP \cdot \exp(\sum w_n \log p_n)$   
**F1** :  $2TP/(2TP + FP + FN)$   
**PMI** :  $\log_2 P(w_1, w_2)/[P(w_1)P(w_2)]$   
**LoRA** :  $h = W_0x + BAx$   
**CoT** :  $P(y|x) = \sum_r P(r|x)P(y|r, x)$

#### Modèles et Dates Clés

**Word2Vec** : Mikolov, Google 2013  
**GloVe** : Pennington, Stanford 2014  
**FastText** : Joulin, Facebook 2016  
**ELMo** : Peters et al., Google 2018 (BiLSTM)  
**Transformer** : Vaswani et al. 2017  
**BERT** : Google 2018 (MLM + NSP)  
**GPT-1/2/3** : OpenAI 2018/2019/2020 (CLM)  
**LoRA** : Hu et al. 2021  
**DPO** : Rafailov et al. 2023

#### Tableau de Synthèse des Paradigmes

| Paradigme            | Mécanisme              | Cas d'usage                                    |
|----------------------|------------------------|------------------------------------------------|
| Self-Supervised      | MLM, CLM               | Comprendre la langue (corpus massif)           |
| Contrastive Learning | Negative Sampling      | Structurer l'espace vectoriel                  |
| Supervisé            | Couples (X, Y)         | Tâches métier (peu de données)                 |
| RLHF                 | PPO/DPO + Reward Model | Comportement humain, sécurité                  |
| RAG                  | Vector Store + LLM     | Connaissances externes, réduire hallucinations |

**Stack Technologique en 2026****Baseline / Prototypage :** VADER, TextBlob, TF-IDF + SVM**Production :** Fine-tuning DistilBERT, MobileBERT (PEFT/LoRA)**Arabe :** AraBERT, CamelTools, MARBERT**RAG :** Bi-Encoder + Vector Store (FAISS, Milvus) + Re-ranker (Cross-Encoder) + RAGAS (évaluation)**LLMs Frontier :** GPT-4, Claude (few-shot, analyses qualitatives profondes)**Bon courage pour l'examen !**

NLP – Master IA FSSM – J.ZAHIR 2026